ean universidad

acreditada
en alta calidad
res. N° 023654 del Mineducación
10/12/21, vigencia 10/12/27

PYTHON PARA PROCESAMIENTO, CALIDAD Y AUTOMATIZACIÓN DE DATOS

Sesión 5 · Proyecto integrador: base de datos, SQL y tablero

De los archivos a una base formal, con datos reales de Colpensiones

Jawy Andrés Romero Pinto · Lunes 9 de noviembre de 2026 · 08:00 – 10:00 a. m.

SOMOS LA U. DEL
EMPRENDIMIENTO
SOSTENIBLE.

Acreditaciones:

Agenda de hoy

Lunes 9 de noviembre · 08:00 – 10:00 a. m.

08:00	**Bienvenida**	Lo que construimos en cuatro sesiones y lo que falta para producción.
08:05	**De archivos a base de datos**	Por qué los archivos sueltos no alcanzan.
08:12	**Datos reales**	Colpensiones y DANE en datos.gov.co: cruzar fuentes con códigos oficiales.
08:20	**Modelo formal**	Dimensiones, hechos, llaves primarias y foráneas, restricciones.
08:28	**Sentencias SQL**	SELECT, WHERE, GROUP BY, JOIN, HAVING, WITH y funciones de ventana.
08:45	**Vistas, tablero y modelos**	Grafana en vivo: llegan meses nuevos, SQL desde Explore y dos modelos.
09:00	**Laboratorio**	Construyes la base, respondes con SQL e integras los modelos (40 min).
09:40	**Hallazgos y cierre**	Tres minutos por equipo y cierre del programa.

ean universidad



01

De archivos a una base de datos

El Excel por correo no escala: la Dirección quiere una sola fuente de verdad.

Lo que construimos y lo que faltaba

Cuatro sesiones de pandas; hoy, el paso a una base formal

Sesión	Pregunta	Hoy se convierte en
1 · Fundamentos	¿Qué hay en el archivo?	Ingesta: los CSV entran a la base
2 · Limpieza	¿Cómo lo corrijo?	Tablas limpias con tipos correctos
3 · Calidad	¿Puedo confiar?	Reglas que la base hace cumplir sola (llaves y CHECK)
4 · Automatización	¿Y el mes que viene?	Un script que recarga la base cada mes
5 · Integrador (hoy)	¿Cómo lo consulta y lo ve toda la organización?	SQL, vistas y un tablero en Grafana

pandas **transforma**; la base de datos **guarda y protege**; SQL **pregunta**; el tablero **muestra**.

Base de datos relacional y SQL

Tablas con reglas, y un idioma estándar para preguntarles

Qué es

Un conjunto de **tablas relacionadas** por columnas comunes, administrado por un motor (SQLite, PostgreSQL, SQL Server, Oracle). **SQL** es el lenguaje estándar para crear, cargar y consultar esas tablas.

Para qué sirve

Una sola fuente de verdad para muchas personas y herramientas a la vez. La base rechaza datos que rompen las reglas, y SQL funciona igual en Power BI, Grafana, Python o Excel.

Analogía con Excel

Un libro con varias hojas unidas por BUSCARV, pero donde la hoja **se niega** a aceptar un código que no existe, y donde cien personas leen al mismo tiempo sin copias.

Cómo manejarlo

Hoy, **SQLite**: viene con Python y es un solo archivo (ideal para Colab). En producción, **PostgreSQL**: el mismo SQL, con usuarios, permisos y servidor. Lo verán en el tablero.

Abstracción desde el código

```
import sqlite3
con = sqlite3.connect("bodega.db")

pd.read_sql("""
  SELECT periodo,
         SUM(valor_aporte)
  FROM fact_aporte
  GROUP BY periodo
""", con)
```

SQL describe **qué** quieres, no **cómo** buscarlo: el motor decide la forma más rápida. Por eso la misma consulta sirve en una laptop y en un servidor con millones de filas.

ean universidad

02

Datos reales y abiertos

Colpensiones y el DANE publican cifras oficiales en datos.gov.co.

Cuatro fuentes oficiales, corte octubre de 2026

Cifras agregadas: no contienen datos personales

Conjunto de datos	Publica	Id	Qué trae
Afiliados por departamento y municipio	Colpensiones	xfsi-rqje	7.299.965 afiliados, por género
Pensionados por departamento y municipio	Colpensiones	hdnf-a76p	1.952.368 pensionados
Pensionados por rango de edad	Colpensiones	np5z-haxm	Edad, salario, género y tipo de pensión
DIVIPOLA · códigos de municipios	DANE	gdxc-w37w	1.122 municipios con latitud y longitud

```
pd.read_csv("https://www.datos.gov.co/resource/xfsi-rqje.csv")
```

Los datos del fondo siguen siendo **ficticios**; los de Colpensiones y el DANE son **reales**. Hoy los unimos.

Los datos reales, en pandas

Una fila por municipio y género

[1]
```
afil_real = pd.read_csv(RUTA +
"reales/colpensiones_afiliados_municipio.csv")
pens_real = pd.read_csv(RUTA +
"reales/colpensiones_pensionados_municipio.csv")
print(f"Afiliados a Colpensiones:  {afil_real['cantidad'].sum():,}")
print(f"Pensionados:               {pens_real['cantidad'].sum():,}")
afil_real.sample(4, random_state=7)
```

```
Afiliados a Colpensiones:  7,299,965
Pensionados:               1,952,368
```

	departamento	municipio	genero	cantidad
1880	PUTUMAYO	ORITO	Femenino	947
651	HUILA	RIVERA	Masculino	987
127	ANTIOQUIA	ZARAGOZA	Masculino	1125
674	LA GUAJIRA	URUMITA	Masculino	352

Qué observar

- **7.299.965** afiliados y **1.952.368** pensionados.
- Una fila por **municipio y género**: datos agregados.
- Los nombres vienen en MAYÚSCULA y sin tildes.

Datos abiertos oficiales: son el **contexto** contra el que se compara la operación del fondo.

Cruzar fuentes con códigos oficiales

El nombre engaña; el código no

Qué es

Un **código estándar** identifica cada entidad sin ambigüedad. En Colombia, el **DIVIPOLA** del DANE da a cada municipio un código de 5 dígitos: Medellín es `05001`, Bogotá `11001`.

Para qué sirve

Unir fuentes que nunca se conocieron. Colpensiones escribe `CALI`; el DANE, `SANTIAGO DE CALI`. Con el código, el cruce es exacto y no depende de tildes ni abreviaturas.

Analogía con Excel

Un BUSCARV por nombre de ciudad que devuelve #N/D en Bogotá, Cali y Cúcuta. Cruzar por código es como usar la cédula en vez del nombre.

Cómo manejarlo

Normaliza el texto, cruza, **cuenta lo que no cruzó** y resuélvelo con una tabla de alias revisada a mano. Lo que no tiene municipio va a un **miembro desconocido** (`00000`): nada se pierde en silencio.

Abstracción desde el código

```
clave = dpto + "|" + municipio

ALIAS_MUNICIPIO = {
  ("bogota d c", "bogota"): "11001",
  ("valle del cauca", "cali"): "76001",
  ("sin ciudad", "sin ciudad"): "00000",
}
```

Un cruce sin verificar es un error silencioso.
Siempre se cuenta lo que **no** cruzó antes de seguir.

El cruce ingenuo pierde a Bogotá

34 filas no encuentran su municipio

[2]
```
clave_dane = divipola["dpto"].map(normalizar_texto) + "|" +
divipola["nom_mpio"].map(normalizar_texto)
clave_colp = (afil_real["departamento"].map(normalizar_texto) + "|"
              + afil_real["municipio"].map(normalizar_texto))
no_cruzan = afil_real[~clave_colp.isin(clave_dane)]
print("Filas sin cruce:", len(no_cruzan), "· afiliados:", f"
{no_cruzan['cantidad'].sum():,}")
no_cruzan.groupby(["departamento", "municipio"])["cantidad"].sum().nlargest(5)
```

```
Filas sin cruce: 34 · afiliados: 2,798,991
departamento        municipio
BOGOTA, D.C.        BOGOTA        1902362
VALLE DEL CAUCA     CALI           532499
BOLIVAR             CARTAGENA      146786
NORTE DE SANTANDER  CUCUTA         118318
SIN CIUDAD          SIN CIUDAD      45405
Name: cantidad, dtype: int64
```

Qué observar

- **2.798.991** afiliados quedan sin código: el **38 %** del país.
- Bogotá, Cali, Cartagena y Cúcuta: nombres distintos en cada fuente.
- Con una tabla de alias y el miembro `00000`, el cruce queda completo.

Sin esta revisión, el tablero diría que Bogotá no tiene afiliados. **La calidad empieza en el cruce.**

03

El modelo formal

Dimensiones, hechos y reglas que la base de datos hace cumplir.

Modelo en estrella: dimensiones y hechos

La forma estándar de una bodega de datos

Qué es

Las tablas de **hechos** guardan lo que pasó y se mide (un aporte, una PQRS). Las de **dimensiones** describen el quién, el dónde y el cuándo (afiliado, municipio, periodo).

Para qué sirve

Cada dato se guarda **una sola vez**: el nombre del municipio vive en `dim_municipio`, no repetido en 15.759 aportes. Las consultas y los tableros quedan simples y rápidos.

Analogía con Excel

La hoja de movimientos (hechos) y las hojas de catálogo (dimensiones) que consultas con BUSCARV. La diferencia: aquí la relación está **declarada** y se verifica.

Cómo manejarlo

Nombres con prefijo (`dim_`, `fact_`), una llave por tabla, hechos con llaves foráneas a cada dimensión. Los datos personales se quedan fuera o seudonimizados (Ley 1581).

Abstracción desde el código

```
fact_aporte
  id_afiliado -> dim_afiliado
  periodo     -> dim_periodo

dim_afiliado
  cod_mpio    -> dim_municipio

dim_municipio
  cod_dpto    -> dim_departamento
```

El modelo es un **contrato**: dice qué existe, cómo se relaciona y qué valores son válidos. Cualquier herramienta que lea la base hereda ese contrato.

Nuestro modelo: 10 tablas, 9 vistas

Datos del fondo y datos reales unidos por el código DIVIPOLA

dim_departamento

33 departamentos + desconocido

dim_municipio

1.122 municipios DANE + 00000

real_afiliados / real_pensionados

Colpensiones, datos reales

cod_mpio une el fondo con los datos reales

dim_periodo

9 meses de 2026

fact_aporte

15.759 aportes

dim_afiliado

3.030, seudonimizados

fact_pqrs → dim_afiliado

calidad_regla

9 reglas medidas

real_pensionados_edad

756 combinaciones reales

fact_pqrs

1.500 PQRS con días hábiles

Llaves y restricciones

Las reglas de calidad de la Sesión 3, escritas dentro de la base

Qué es

`PRIMARY KEY`: identifica cada fila, única y obligatoria. `REFERENCES` (llave **foránea**): el valor debe existir en otra tabla. `NOT NULL`: obligatorio. `CHECK`: una condición que todo valor cumple.

Para qué sirve

Que los errores **no entren**. La regla 7 de la Sesión 3 (el afiliado existe) ya no hay que medirla después: la base rechaza el aporte huérfano en el momento de insertarlo.

Analogía con Excel

Validación de datos de Excel (lista desplegable, número entre 1 y 12), pero obligatoria para todos, sin la opción de "continuar de todos modos".

Cómo manejarlo

Declara las llaves en el `CREATE TABLE`. Carga primero las dimensiones y luego los hechos. En SQLite, activa `PRAGMA foreign_keys = ON`: por defecto no las revisa.

Abstracción desde el código

```
CREATE TABLE fact_aporte (
  id_aporte INTEGER PRIMARY KEY,
  id_afiliado TEXT NOT NULL
    REFERENCES dim_afiliado(id_afiliado),
  periodo TEXT NOT NULL
    REFERENCES dim_periodo(periodo),
  valor_aporte BIGINT NOT NULL
);
```

Medir la calidad está bien; **impedir** que el error entre es mejor. Las restricciones son reglas de calidad que nunca se olvidan de correr.

La estructura, consultada a la propia base

La base sabe cómo está construida

[1]
```
print("Columnas de fact_aporte:")
print(sql('SELECT name, type, "notnull", pk FROM
pragma_table_info("fact_aporte")').to_string(index=False))
print("\nLlaves foráneas: de dónde viene cada valor")
sql('SELECT "from" AS columna, "table" AS tabla_destino, "to" AS llave '
    'FROM pragma_foreign_key_list("fact_aporte")')
```

```
Columnas de fact_aporte:
             name    type  notnull  pk
        id_aporte INTEGER        0   1
      id_afiliado    TEXT        1   0
          periodo    TEXT        1   0
              ibc  BIGINT        1   0
     valor_aporte  BIGINT        1   0
       fecha_pago    DATE        0   0
         operador    TEXT        0   0
aporte_incorrecto INTEGER        1   0

Llaves foráneas: de dónde viene cada valor
```

	columna	tabla_destino	llave
0	periodo	dim_periodo	periodo
1	id_afiliado	dim_afiliado	id_afiliado

Qué observar

- `pk = 1`: **id_aporte** es la llave primaria.
- `notnull = 1`: afiliado, periodo y valores son obligatorios.
- Dos llaves foráneas: a **dim_periodo** y a **dim_afiliado**.

El modelo no está en un documento aparte: está **dentro de la base**, y cualquier herramienta lo puede leer.

La base se defiende sola

Tres intentos de meter datos malos

[4]
```
intentos = {
    "aporte huérfano": "INSERT INTO fact_aporte (id_aporte, id_afiliado, periodo,
ibc, "
                       "valor_aporte, aporte_incorrecto) "
                       "VALUES (999999, 'AF899999', '2026-01', 2000000, 320000, 0)",
    "género inválido": "UPDATE dim_afiliado SET genero = 'X' WHERE id_afiliado =
'AF100026'",
    "mes 13":          "INSERT INTO dim_periodo VALUES ('2026-13', '2026-13-01',
2026, 13, 5)",
}
for nombre, sentencia in intentos.items():
    try:
        con.execute(sentencia)
        print(f"{nombre}: se aceptó")
    except sqlite3.IntegrityError as e:
        print(f"{nombre}: RECHAZADO -> {e}")
```

```
aporte huérfano: RECHAZADO -> FOREIGN KEY constraint failed
género inválido: RECHAZADO -> CHECK constraint failed: genero IN ('F', 'M')
mes 13: RECHAZADO -> CHECK constraint failed: mes BETWEEN 1 AND 12
```

Qué observar

- Aporte de un afiliado que no existe: **FOREIGN KEY**.
- Género X: **CHECK** `genero IN ('F', 'M')`.
- Mes 13: **CHECK** `mes BETWEEN 1 AND 12`.

En Excel, los tres habrían entrado sin aviso. Aquí **no hay forma** de guardarlos.

ean universidad

04

Sentencias SQL

Preguntas de negocio en un idioma que entienden todas las herramientas.

Anatomía de una consulta

Siempre el mismo orden; se lee casi como una frase

Cláusula	Qué hace	En Excel / pandas
SELECT	Qué columnas o cálculos quiero ver	Las columnas que dejas visibles
FROM … JOIN … ON	De qué tablas, y cómo se unen	BUSCARV · merge()
WHERE	Qué **filas** entran	Autofiltro · df[mascara]
GROUP BY	Cómo agrupar	Filas de la tabla dinámica · groupby()
HAVING	Qué **grupos** quedan	Filtro sobre la tabla dinámica
ORDER BY … LIMIT	Orden y cuántas filas mostrar	Ordenar de mayor a menor · head()

Se escribe en ese orden, pero el motor lo ejecuta así: **FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**.

SELECT, WHERE, ORDER BY y LIMIT

Los municipios de Antioquia más al norte

[5]

```
sql("""
SELECT cod_mpio, municipio, latitud
FROM dim_municipio
WHERE cod_dpto = '05'
ORDER BY latitud DESC
LIMIT 5
""")
```

	cod_mpio	municipio	latitud
0	05051	ARBOLETES	8.849317
1	05659	SAN JUAN DE URABÁ	8.758964
2	05490	NECOCLÍ	8.434526
3	05665	SAN PEDRO DE URABÁ	8.276884
4	05495	NECHÍ	8.094129

Qué observar

- `WHERE cod_dpto = '05'`: solo Antioquia.
- `ORDER BY latitud DESC`: de norte a sur.
- `LIMIT 5`: las cinco primeras.

La latitud ya es un número: la conversión de la coma decimal se hizo **antes** de cargar.

GROUP BY: la tabla dinámica de SQL

Recaudo y aportantes por mes

[6]

```
sql("""
SELECT periodo,
       SUM(valor_aporte)           AS recaudo,
       COUNT(DISTINCT id_afiliado) AS aportantes
FROM fact_aporte
GROUP BY periodo
ORDER BY periodo
""")
```

	periodo	recaudo	aportantes
0	2026-01	961713352	1757
1	2026-02	955089877	1744
2	2026-03	964307712	1757
3	2026-04	967001768	1762
4	2026-05	957157676	1748
5	2026-06	960364484	1751
6	2026-07	970357871	1772
7	2026-08	953646598	1736
8	2026-09	953035897	1732

Qué observar

- `SUM(valor_aporte)`: el recaudo de cada mes.
- `COUNT(DISTINCT id_afiliado)`: aportantes únicos.
- Total ene-sep: **$8.642.675.235**, sin los 72 huérfanos.

Las mismas cifras de la Sesión 4, ahora salen de **una consulta guardable**.

JOIN: unir tablas por su llave

La razón de existir de las llaves foráneas

Qué es

`JOIN tabla ON a.llave = b.llave` une cada fila con su pareja en otra tabla. `LEFT JOIN` conserva además las filas que **no** tienen pareja (con vacíos).

Para qué sirve

Responder preguntas que cruzan entidades: afiliados reales **por departamento** necesita unir hechos → municipio → departamento.

Analogía con Excel

BUSCARV encadenados: del municipio traigo el código del departamento, y de ahí su nombre. En pandas es `merge()`.

Cómo manejarlo

Usa alias cortos (`r`, `m`, `d`). Une siempre por la **llave**, no por el nombre. Verifica el total antes y después: si crece, la llave no era única.

Abstracción desde el código

```
SELECT d.departamento,
       SUM(r.cantidad)
FROM real_afiliados r
JOIN dim_municipio m
  ON m.cod_mpio = r.cod_mpio
JOIN dim_departamento d
  ON d.cod_dpto = m.cod_dpto
GROUP BY d.departamento
```

El modelo hace que los JOIN sean seguros: como las llaves están declaradas, cada aporte tiene **exactamente** un afiliado y cada municipio un departamento.

JOIN sobre datos reales

Los departamentos con más afiliados a Colpensiones

[7]

```
sql("""
SELECT d.departamento, SUM(r.cantidad) AS afiliados
FROM real_afiliados r
JOIN dim_municipio m    ON m.cod_mpio = r.cod_mpio
JOIN dim_departamento d ON d.cod_dpto = m.cod_dpto
GROUP BY d.departamento
ORDER BY afiliados DESC
LIMIT 6
""")
```

	departamento	afiliados
0	BOGOTÁ, D.C.	1902362
1	ANTIOQUIA	1266328
2	VALLE DEL CAUCA	801018
3	SANTANDER	390583
4	CUNDINAMARCA	350076
5	ATLÁNTICO	307020

Qué observar

- Tres tablas unidas por dos llaves.
- Bogotá: **1,9 millones**; Antioquia: **1,27 millones**.
- Solo funciona porque Bogotá ya tiene su código `11001`.

Sin el alias de la parte 2, Bogotá **no aparecería** en este ranking.

Un indicador real: pensionados por cada 100 afiliados

Desde la vista v_real_departamento

[8]

```
sql("""
SELECT departamento, afiliados, pensionados,
       pensionados_por_100_afiliados AS pens_x_100
FROM v_real_departamento
WHERE afiliados > 50000
ORDER BY pens_x_100 DESC
LIMIT 6
""")
```

	departamento	afiliados	pensionados	pens_x_100
0	RISARALDA	161810	56411	34.9
1	CALDAS	163053	56309	34.5
2	VALLE DEL CAUCA	801018	276227	34.5
3	ANTIOQUIA	1266328	382756	30.2
4	CUNDINAMARCA	350076	102474	29.3
5	BOGOTÁ, D.C.	1902362	553668	29.1

Qué observar

- Risaralda: **34,9** pensionados por cada 100 afiliados.
- Eje Cafetero y Valle encabezan la lista.
- Bogotá: 29,1. Promedio nacional: **26,7**.

Una pregunta de **sostenibilidad** respondida con datos oficiales y una consulta de 6 líneas.

HAVING: filtrar grupos

Temas de PQRS por debajo de la meta del 90 %

[9]
```
sql("""
SELECT tema,
       COUNT(*)                          AS radicadas,
       ROUND(100.0 * AVG(a_tiempo), 1) AS pct_a_tiempo
FROM fact_pqrs
GROUP BY tema
HAVING 100.0 * AVG(a_tiempo) < 90
ORDER BY pct_a_tiempo
""")
```

	tema	radicadas	pct_a_tiempo
0	Devolución de aportes	242	87.2
1	Atención al usuario	215	88.0
2	Corrección de datos	273	88.5
3	Certificados	262	89.4
4	Pensión de vejez	244	89.4

Qué observar

- `AVG(a_tiempo)`: proporción respondida a tiempo.
- `HAVING`: solo los temas bajo la meta.
- Peor tema: **Devolución de aportes (87,2 %)**.

WHERE no sirve aquí: el % solo existe **después** de agrupar.

WITH y funciones de ventana

Consultas por pasos y cálculos que miran la fila vecina

Qué es

`WITH nombre AS (...)` crea una tabla temporal con nombre (CTE) dentro de la consulta. Una **función de ventana** como `LAG(x) OVER (ORDER BY periodo)` trae el valor de otra fila sin agrupar.

Para qué sirve

Escribir consultas complejas paso a paso y legibles. Calcular variaciones mes a mes, acumulados o rankings sin perder el detalle.

Analogía con Excel

WITH es una hoja auxiliar con un cálculo intermedio. LAG es la fórmula `=B3-B2` que compara cada fila con la de arriba.

Cómo manejarlo

Un paso por CTE, con nombre de negocio. En las ventanas, el `ORDER BY` dentro de `OVER` define qué es "la fila anterior".

Abstracción desde el código

```
WITH mensual AS (
  SELECT periodo,
         SUM(valor_aporte) AS recaudo
  FROM fact_aporte GROUP BY periodo
)
SELECT periodo, recaudo,
  LAG(recaudo) OVER (ORDER BY periodo)
FROM mensual
```

Así se escriben las consultas de los tableros reales: **pasos con nombre**, igual que las funciones de la Sesión 4.

WITH + LEFT JOIN: la lista de mora

Activos con 3 o más meses sin aportar, por regional

[10]

```
sql("""
WITH meses AS (
    SELECT id_afiliado, COUNT(DISTINCT periodo) AS meses_con_aporte
    FROM fact_aporte
    GROUP BY id_afiliado
)
SELECT f.regional, COUNT(*) AS afiliados_en_mora
FROM dim_afiliado f
LEFT JOIN meses x ON x.id_afiliado = f.id_afiliado
WHERE f.vigente = 1 AND f.estado = 'Activo'
  AND 9 - COALESCE(x.meses_con_aporte, 0) >= 3
GROUP BY f.regional
ORDER BY afiliados_en_mora DESC
""")
```

	regional	afiliados_en_mora
0	Centro	20
1	Antioquia	9
2	Occidente	7
3	Caribe	7
4	Eje Cafetero	6
5	Oriente	5

Qué observar

- La CTE cuenta los meses con aporte de cada afiliado.
- `LEFT JOIN`: incluye a quien no aportó **ningún** mes.
- **54** en mora; la regional Centro concentra **20**.

Con un JOIN normal, quien nunca aportó **desaparecería** de la lista: justo el caso más grave.

Función de ventana: variación mes a mes

LAG compara cada mes con el anterior

[11]

```
sql("""
WITH mensual AS (
    SELECT periodo, SUM(valor_aporte) AS recaudo
    FROM fact_aporte GROUP BY periodo
)
SELECT periodo, recaudo,
       ROUND(100.0 * (recaudo - LAG(recaudo) OVER (ORDER BY periodo))
             / LAG(recaudo) OVER (ORDER BY periodo), 2) AS variacion_pct
FROM mensual
ORDER BY periodo
""")
```

	periodo	recaudo	variacion_pct
0	2026-01	961713352	NaN
1	2026-02	955089877	-0.69
2	2026-03	964307712	0.97
3	2026-04	967001768	0.28
4	2026-05	957157676	-1.02
5	2026-06	960364484	0.34
6	2026-07	970357871	1.04
7	2026-08	953646598	-1.72
8	2026-09	953035897	-0.06

Qué observar

- Enero no tiene mes anterior: NaN.
- Mayor alza: julio (**+1,04 %**).
- Mayor caída: agosto (**-1,72 %**).

El recaudo es **estable**: ningún mes se mueve más del 2 %.

ean universidad

05

Vistas y tablero

La capa que ve la Dirección, actualizada sola.

Vistas: consultas guardadas con nombre

La capa "oro" del modelo

Qué es

Una **vista** (`CREATE VIEW`) es una consulta guardada en la base. Se consulta como una tabla, pero se calcula cada vez con los datos del momento.

Para qué sirve

Que todos usen **la misma definición** de cada indicador. El tablero solo lee vistas: si cambia una regla, se cambia la vista y se actualizan todos los gráficos.

Analogía con Excel

Una tabla dinámica conectada a la fuente que nadie puede romper: cada quien la consulta, nadie la edita.

Cómo manejarlo

Prefijo `v_`, una vista por pregunta de negocio, y sin datos personales: `v_mora` muestra el documento seudonimizado, nunca el nombre.

Abstracción desde el código

```
CREATE VIEW v_pqrs_canal AS
SELECT canal,
       COUNT(*) AS radicadas,
       ROUND(100.0 * SUM(a_tiempo)
             / COUNT(a_tiempo), 1)
         AS pct_a_tiempo
FROM fact_pqrs
GROUP BY canal;
```

Python carga, la base guarda, **las vistas definen los indicadores** y el tablero solo los muestra.

Las vistas, consultadas

Un indicador = una vista

[12]
```
print(sql("SELECT sql FROM sqlite_master WHERE name = 'v_pqrs_canal'")["sql"][0])
sql("SELECT * FROM v_kpi_fondo")
```

```
CREATE VIEW v_pqrs_canal AS
SELECT canal,
       COUNT(*)                          AS radicadas,
       ROUND(CAST(100.0 * SUM(a_tiempo) / COUNT(a_tiempo) AS NUMERIC), 1) AS pct_a_tiempo
FROM fact_pqrs
GROUP BY canal
```

	recaudo_total	pqrs_radicadas	pct_pqrs_a_tiempo	dias_habiles_promedio	activos_en_mora	peor_regla_pct
0	8642675235	1500	88.9	8.3	54	96.5

Qué observar

- La base guarda el SQL de cada vista.
- `v_kpi_fondo`: los 6 indicadores de la Dirección en una fila.
- Recaudo **$8.643 M** · PQRS a tiempo **88,9 %** · mora **54**.

Grafana, Power BI o Excel leen **estas mismas vistas**: un solo número para toda la organización.

La arquitectura completa

Del CSV al tablero, sin pasos manuales

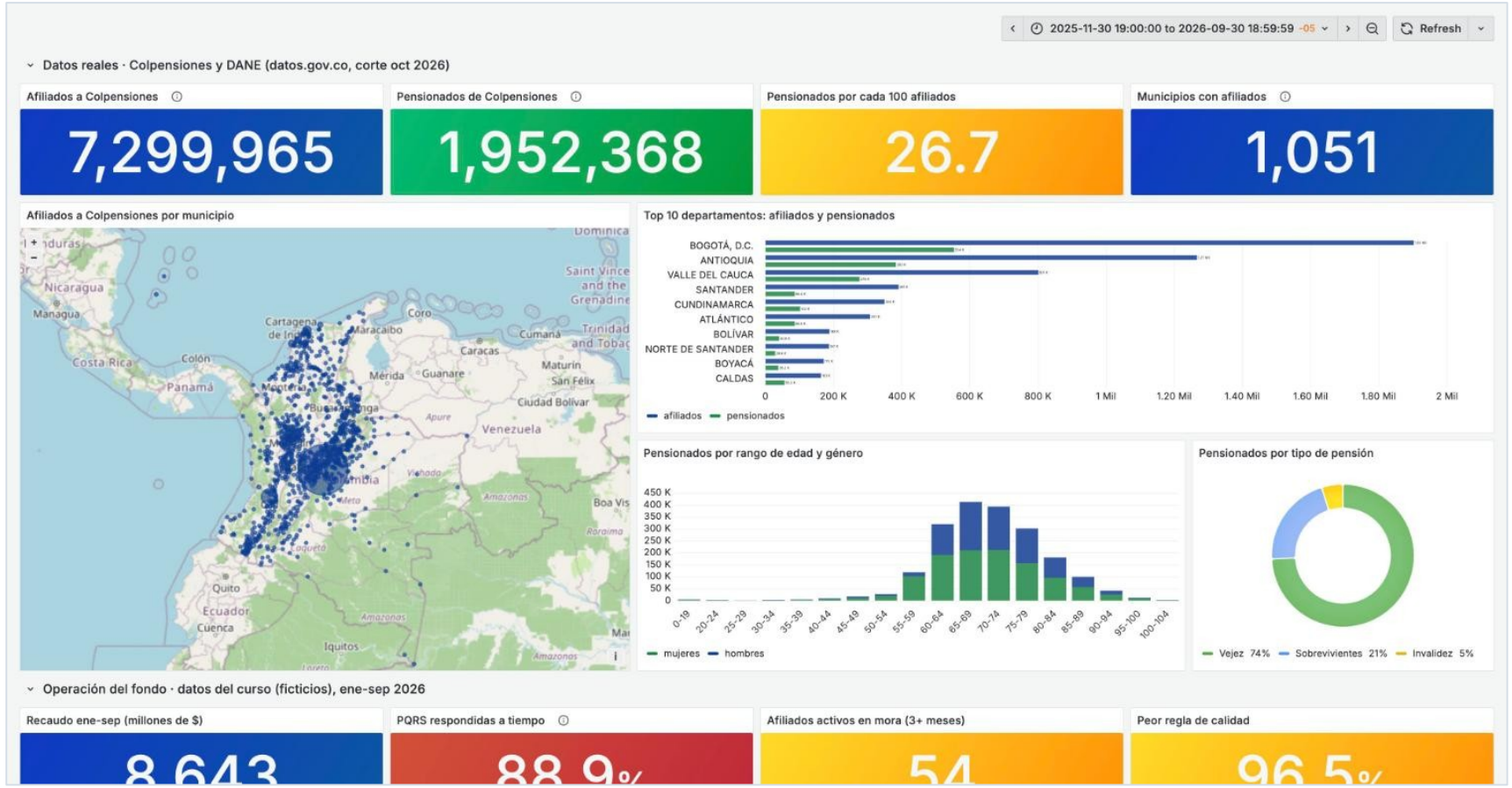



```
python pipeline_bodega.py --datos ../datos --postgres tablero/init   ·   docker compose up -d
```

El mismo SQL corre en **SQLite** (Colab, un archivo) y en **PostgreSQL** (servidor). Cambiar de motor no cambia las preguntas.

Tablero en Grafana: datos reales

PostgreSQL + Grafana, levantados con docker compose



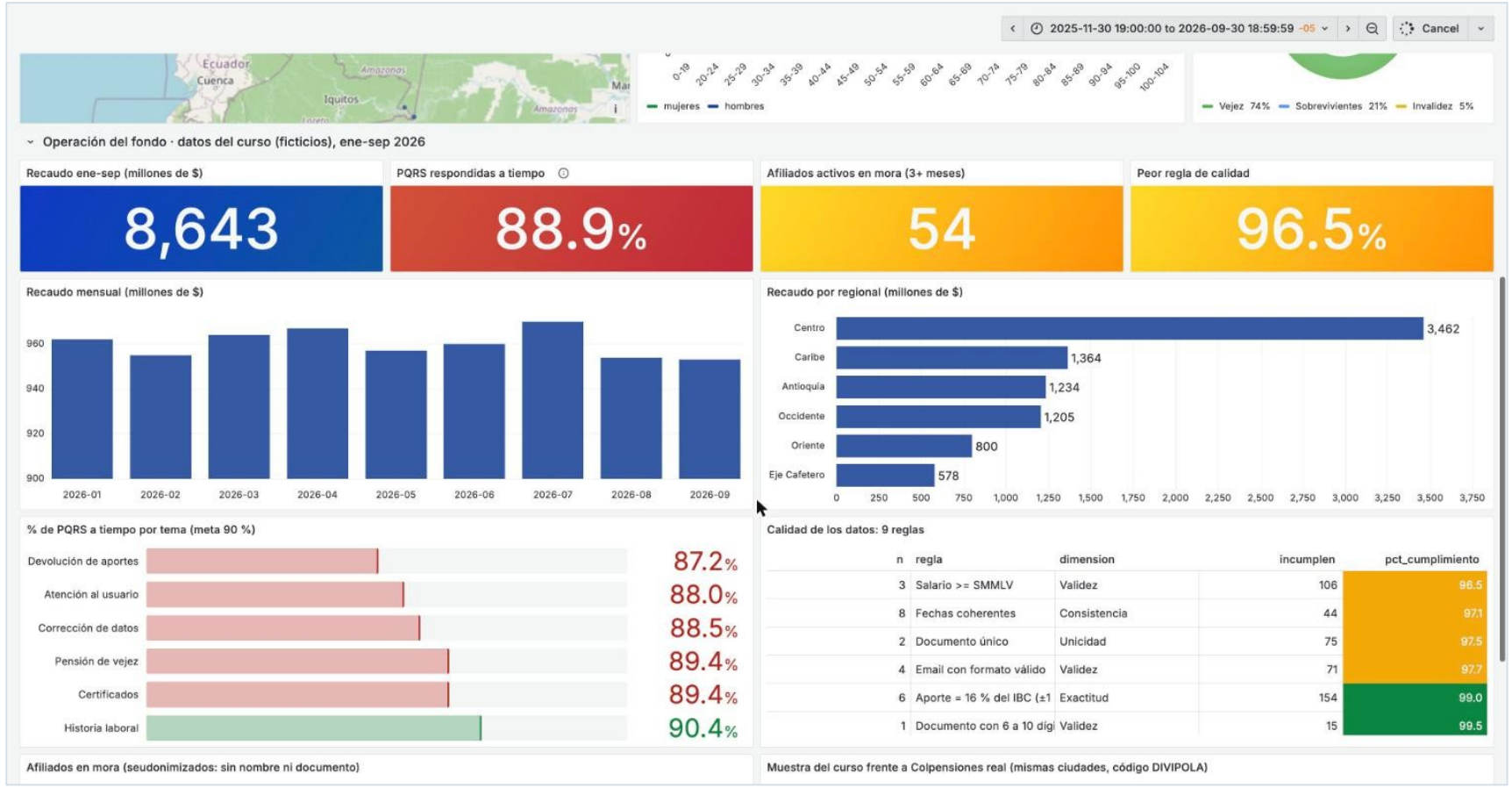
Qué observar

- Totales oficiales de Colpensiones.
- **Mapa** con los 1.051 municipios con afiliados (coordenadas DANE).
- Pirámide de pensionados: el grueso entre **60 y 79 años**.
- Vejez: **74 %** de las pensiones.

Cada panel es **una consulta SQL** sobre las vistas. Clic en el título → Editar, y se ve el SQL.

Tablero en Grafana: operación del fondo

Indicadores con semáforo por umbral



Qué observar

- PQRS a tiempo **88,9 %** en rojo: meta 90 %.
- Solo *Historia laboral* supera la meta.
- Calidad: la peor regla (salario ≥ SMMLV) en **96,5 %**.
- Mora seudonimizada: sin nombre ni documento.

Los umbrales vuelven el tablero una **alarma**: rojo significa que alguien debe actuar.

Automatización completa: llega un archivo y todo se actualiza

Nadie abre Excel; el tablero cambia solo

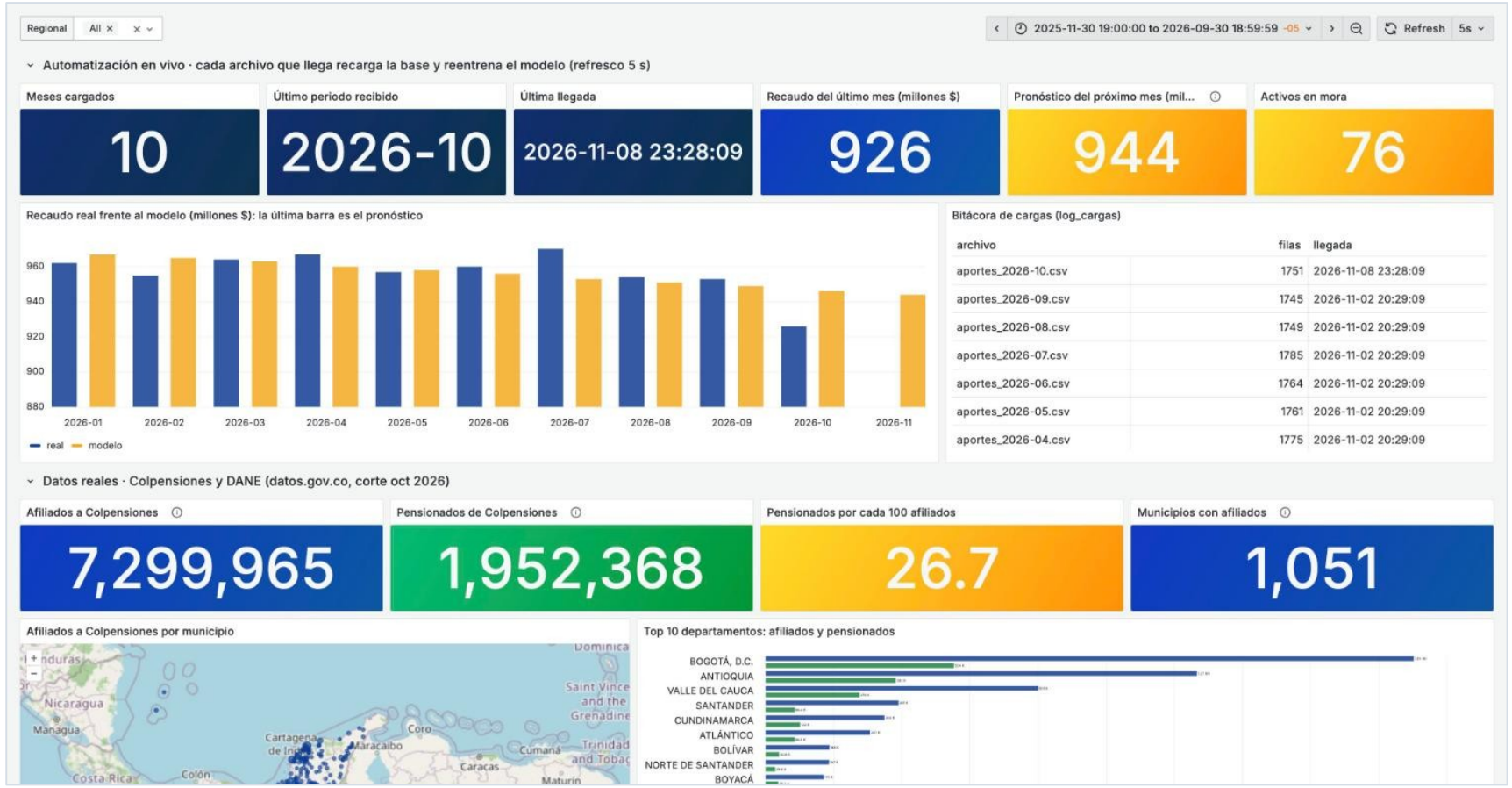



```
python simular_llegada.py --un-mes        # llega el siguiente mes
python simular_llegada.py --reiniciar     # vuelve a enero-septiembre
```

En producción, el simulador lo reemplaza el **Programador de tareas** o `cron`: el día 5 a las 6:00 a. m. corre el pipeline y, cuando la Dirección abre el tablero, el mes ya está.

Automatización en vivo en Grafana

Llegó octubre: la fila superior cambió sola



Qué observar

- **Meses cargados** pasa de 9 a 10.
- Octubre: **$926 M**, la caída más fuerte del año.
- El modelo baja el pronóstico de noviembre a **$944 M**.
- La **bitácora** muestra cada archivo y su hora de llegada.

El tablero no se edita: **se alimenta**. Cada carga deja rastro en `log_cargas`.

El código: llega un mes nuevo

simular_llegada.py hace el papel del operador que entrega el archivo

tablero/simular_llegada.py

```
def llega_un_mes():
    meses = pb.meses_disponibles(VIVO)            # 1
    mes = max(meses) + 1                          # 9 -> 10
    if mes > 12:
        return False                              # ya llegó diciembre
    _, pqrs, aportes, _ = pb.cargar_curso(VIVO)
    aportes = pb.limpiar_aportes(aportes)
    nuevo, nuevas = pb.simular_mes(aportes, pqrs, mes)   # 2

    archivo = f"{VIVO}aportes/aportes_2026-{mes:02d}.csv"
    nuevo.to_csv(archivo, index=False)            # 3
    nuevas.to_csv(VIVO + "pqrs.csv", mode="a", header=False, index=False)

    pb.main(VIVO, SALIDA, carpeta_vivo=AQUI)      # 4
    return True
# --cada 20: un mes cada 20 segundos, hasta diciembre
while llega_un_mes():                             # 5
    time.sleep(a.cada)
```

Qué hace cada parte

1. Mira qué archivos `aportes_2026-*.csv` hay y calcula el mes siguiente.
2. Fabrica octubre a partir de septiembre, con algo menos de aportantes.
3. Deja el CSV en la carpeta, como lo haría el operador PILA.
4. Llama al **mismo** `main` del pipeline: nada se hace a mano.
5. Repite solo. En producción lo hace `cron` o el Programador de tareas.

> En producción los pasos 2 y 3 no existen: el archivo **llega**. El resto es idéntico.

El código: del pipeline a Grafana, sin dejar la base a medias

main() encadena los pasos de todo el programa; la carga es todo o nada

pipeline_bodega.py (resumido)

```
def main(ruta_datos, salida, carpeta_vivo=None):
    afiliados_raw, pqrs, aportes_raw, catalogo = cargar_curso(ruta_datos)
    afiliados = limpiar_afiliados(afiliados_raw, catalogo)        # 1
    aportes = limpiar_aportes(aportes_raw)
    calidad = evaluar_calidad(afiliados, aportes, pqrs, ...)      # 2
    tablas = construir_modelo(afiliados, aportes, pqrs, ...)      # 3
    tablas = agregar_modelos(tablas)                              # 4
    if carpeta_vivo:
        cargar_postgres_vivo(tablas, carpeta_vivo)                # 5

def cargar_postgres_vivo(tablas, carpeta_tablero):
    partes = ["BEGIN;", "TRUNCATE " + ", ".join(tablas) + " CASCADE;"]
    for nombre, df in tablas.items():
        partes.append(f"COPY {nombre} FROM STDIN (FORMAT csv, HEADER);")
        partes.append(df.to_csv(index=False) + "\\.")
    partes.append("COMMIT;")                                      # 6
    subprocess.run(["docker", "compose", "exec", "-T", "bodega", "psql", ...],
                   input="\n".join(partes), text=True, check=True)
```

Qué hace cada parte

1. Limpiar: el código de las Sesiones 2 y 4.
2. Validar: las 9 reglas de calidad de la Sesión 3.
3. Modelo formal: dimensiones, hechos y seudónimos.
4. Reentrenar los dos modelos con el mes nuevo.
5. Cargar al PostgreSQL que ya está corriendo.
6. `BEGIN … COMMIT`: si algo falla, no se guarda nada.

`COPY` carga cada tabla de un solo golpe, mucho más rápido que un `INSERT` por fila.

Integrar un modelo a la bodega

El resultado del modelo es una tabla más

Qué es

Un **modelo** aprende una relación de los datos y la usa para estimar. Aquí: una **regresión lineal** para pronosticar el recaudo y una **regresión log-log** que estima los pensionados esperados de cada municipio.

Para qué sirve

Anticipar (cuánto recaudaremos) y detectar lo inusual (municipios fuera de lo esperado). Si el modelo escribe en la base, **cualquier herramienta** usa sus resultados.

Analogía con Excel

La línea de tendencia de un gráfico de Excel con su pronóstico, pero recalculada sola cada mes y guardada en una hoja que todos consultan.

Cómo manejarlo

Entrena con `scikit-learn`, guarda predicciones y métricas (`pred_recaudo`, `modelo_metricas`) y **reentrena** con cada carga. Un modelo sin métrica no se publica.

Abstracción desde el código

```
modelo = LinearRegression()
modelo.fit(X, y)
pred.to_sql("pred_recaudo", con)
-- en SQL / Grafana:
SELECT periodo, recaudo_modelo
FROM pred_recaudo
```

El modelo deja de ser un experimento cuando sus resultados viven en la base, se actualizan solos y alguien los consulta.

El código: pronóstico del recaudo

Regresión lineal sobre el tiempo, reentrenada en cada carga

pipeline_bodega.py · pronosticar_recaudo (resumido)

```
def pronosticar_recaudo(fact_aporte):
    mensual = (fact_aporte.groupby("periodo", as_index=False)
                          ["valor_aporte"].sum())
    mensual["t"] = np.arange(len(mensual))                # 1

    modelo = LinearRegression().fit(mensual[["t"]],       # 2
                                    mensual["valor_aporte"])
    mensual["recaudo_modelo"] = modelo.predict(mensual[["t"]])

    pron = modelo.predict(pd.DataFrame({"t": [len(mensual)]}))[0]   # 3

    error = (mensual["valor_aporte"] - mensual["recaudo_modelo"]).std()
    inferior, superior = pron - 1.96 * error, pron + 1.96 * error   # 4

    mape = (abs(mensual["valor_aporte"] - mensual["recaudo_modelo"])
            / mensual["valor_aporte"]).mean() * 100      # 5
    return tabla, {"metrica": "MAPE %", "valor": mape}
# agregar_modelos(): tablas["pred_recaudo"] = tabla     # 6
```

Qué hace cada parte

1. El tiempo como número: enero = 0, …, septiembre = 8.
2. Ajusta la recta que mejor pasa por los recaudos.
3. Pronostica t = 9: el mes que todavía no llega.
4. Rango probable: ±1,96 veces el error típico.
5. MAPE: error promedio en %. Hoy **0,48 %**.
6. El resultado se guarda como tabla; Grafana lo lee.

Cuando llega octubre, `t` va hasta 9 y el modelo se ajusta solo: por eso el pronóstico cambia en vivo.

El código: pensionados esperados con datos reales

Regresión log-log: cuánto 'debería' tener cada municipio

pipeline_bodega.py · modelo_pensionados_esperados (resumido)

```
af = real_afiliados.groupby("cod_mpio")["cantidad"].sum().rename("afiliados")
pe = real_pensionados.set_index("cod_mpio")["cantidad"].rename("pensionados")
df = pd.concat([af, pe], axis=1).dropna()                    # 1
df = df[(df.index != "00000") & (df["afiliados"] > 0) & (df["pensionados"] > 0)]

X, y = np.log(df[["afiliados"]]), np.log(df["pensionados"])  # 2
modelo = LinearRegression().fit(X, y)                        # 3

df["pensionados_esperados"] = np.exp(modelo.predict(X)).round()
df["desviacion_pct"] = (df["pensionados"]                              # 4
                        / df["pensionados_esperados"] - 1) * 100
r2 = modelo.score(X, y)                                      # 5

-- Lo que no encaja, en SQL (panel Modelos de Grafana)
SELECT municipio, afiliados, pensionados, desviacion_pct
FROM modelo_pensionados JOIN dim_municipio USING (cod_mpio)
WHERE afiliados >= 20000
ORDER BY desviacion_pct LIMIT 5;                             -- 6
```

Qué hace cada parte

1. Une afiliados y pensionados por código DIVIPOLA.
2. Logaritmos: Bogotá y un pueblo pequeño caben en la misma recta.
3. Aprende cuántos pensionados trae cada afiliado.
4. Esperados y desviación en % para cada municipio.
5. R^2 = **0,829** en 1.042 municipios.
6. Primero sale ARMENIA (ANT): **−99,7 %**.

El modelo no corrige nada: **señala** qué revisar. La revisión la hace una persona.

El modelo encontró un error en los datos oficiales

Municipios con menos pensionados de lo esperado (datos reales)

[1]
```
sql("""
SELECT m.municipio, d.departamento, p.afiliados, p.pensionados,
       p.pensionados_esperados AS esperados, p.desviacion_pct
FROM modelo_pensionados p
JOIN dim_municipio m    ON m.cod_mpio = p.cod_mpio
JOIN dim_departamento d ON d.cod_dpto = m.cod_dpto
WHERE p.afiliados >= 20000
ORDER BY p.desviacion_pct ASC
LIMIT 5
""")
```

	municipio	departamento	afiliados	pensionados	esperados	desviacion_pct
0	ARMENIA	ANTIOQUIA	61555	43	15272	-99.7
1	QUIBDÓ	CHOCÓ	33011	2276	7005	-67.5
2	RIOHACHA	LA GUAJIRA	32501	2839	6870	-58.7
3	FLORENCIA	CAQUETÁ	33498	3308	7135	-53.6
4	BOGOTÁ, D.C.	BOGOTÁ, D.C.	1902362	553668	1116065	-50.4

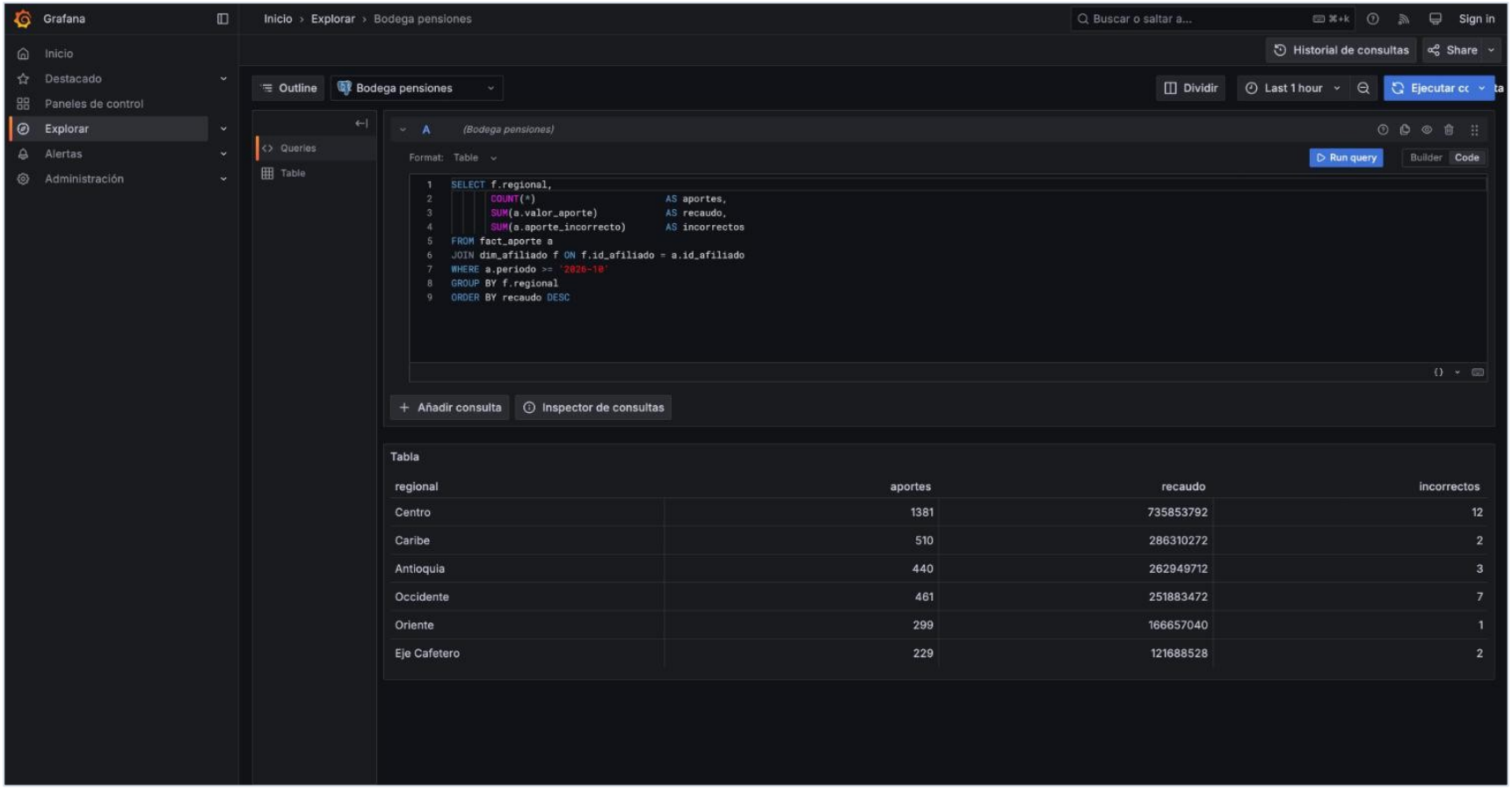
Qué observar

- "ARMENIA" en **Antioquia**: 61.555 afiliados y **43** pensionados.
- Armenia, la capital del **Quindío**, no aparece entre los afiliados.
- Sus afiliados quedaron registrados en el municipio equivocado.

R^2 = **0,829**: el modelo explica bien el patrón, y por eso lo que **no** encaja salta a la vista.

SQL directo desde Grafana

Explore: escribir una sentencia, ejecutarla y descargar el resultado



Qué observar

- Menú **Explorar** → modo **Code** → SQL → *Run query*.
- **Inspector de consultas** → Data → *Download CSV*.
- Grafana usa un usuario de **solo lectura**: un `DELETE` responde *permission denied*.
- `extraer_sql.py`: la misma consulta por API, a CSV.

Consultar sí, modificar no: **el tablero nunca puede dañar la base**.

Las sentencias de la demo en Grafana

Para copiar en Explorar → Code → Run query

Grafana · Explorar (SQL)

```
-- Mora de una regional (vista)
SELECT * FROM v_mora WHERE regional = 'Centro';                    -- 1

-- Qué archivos llegaron y cuándo
SELECT archivo, periodo, filas, llegada
FROM log_cargas ORDER BY llegada DESC;                            -- 2

-- El modelo junto al dato real
SELECT periodo, recaudo_real, recaudo_modelo, limite_inferior, limite_superior
FROM pred_recaudo ORDER BY periodo;                               -- 3

SELECT modelo, metrica, valor, entrenado FROM modelo_metricas;    -- 4

DELETE FROM fact_aporte;                                          -- 5
-- ERROR: permission denied for table fact_aporte

-- Por qué (init/04_lector.sql): Grafana entra con este usuario
GRANT SELECT ON ALL TABLES IN SCHEMA public TO lector_grafana;    -- 6
```

Qué hace cada parte

1. La mora de la regional Centro, sin nombres ni documentos.
2. La bitácora: octubre aparece en cuanto llega.
3. Real contra modelo; el último mes trae el rango probable.
4. MAPE 0,48 % y R^2 0,829, con la hora de entrenamiento.
5. Borrar desde el tablero: **rechazado**.
6. Solo `SELECT`: consultar sí, modificar no.

Cualquier resultado se descarga: **Inspector de consultas → Data → Download CSV**.

Extraer datos por API: extraer_sql.py

La misma consulta, desde un script o un notebook

tablero/extraer_sql.py (resumido)

```
GRAFANA = "http://localhost:3300"

def consultar(sql):
    cuerpo = {"queries": [{"refId": "A", "datasource": {"uid": "bodega"},   # 1
                           "rawSql": sql, "format": "table"}]}
    req = urllib.request.Request(f"{GRAFANA}/api/ds/query",                 # 2
                                 data=json.dumps(cuerpo).encode(),
                                 headers={"Content-Type": "application/json"})
    respuesta = json.load(urllib.request.urlopen(req))
    marco = respuesta["results"]["A"]["frames"][0]                          # 3
    columnas = [c["name"] for c in marco["schema"]["fields"]]
    filas = list(zip(*marco["data"]["values"]))                             # 4
    return columnas, filas

# En la terminal:                                                           # 5
#   python3 extraer_sql.py "SELECT * FROM v_kpi_fondo" --csv kpi.csv
# En pandas:
#   pd.DataFrame(filas, columns=columnas)
```

Qué hace cada parte

1. La consulta va dirigida a la fuente `bodega` de Grafana.
2. Se envía por HTTP, igual que lo hace un panel.
3. Grafana responde en JSON: un marco con columnas.
4. Grafana entrega columnas; `zip` las vuelve filas.
5. Se guarda en CSV o se pasa a pandas.

El script no conoce la clave de la base: pasa por Grafana con el usuario de **solo lectura**.

De Excel a SQL: equivalencias de hoy

Lo que ya sabes hacer, ahora en una base de datos

En Excel	En SQL
Hoja con encabezados	`CREATE TABLE` con tipos de dato
Columna de códigos únicos	`PRIMARY KEY`
BUSCARV que nunca da #N/D	`REFERENCES` (llave foránea) y `JOIN … ON`
Validación de datos	`NOT NULL` y `CHECK (…)`
Autofiltro	`WHERE`
Tabla dinámica	`GROUP BY` con `SUM`, `COUNT`, `AVG`
Filtro sobre la tabla dinámica	`HAVING`
Hoja auxiliar de cálculo	`WITH nombre AS (…)`
=B3-B2 (comparar con la fila de arriba)	`LAG(x) OVER (ORDER BY …)`
Gráfico conectado a la tabla dinámica	`CREATE VIEW` + panel de Grafana
Línea de tendencia con pronóstico	`LinearRegression` → tabla `pred_recaudo`

LAB

40 min

Laboratorio · Sesión 5

Abre **PY-EAN_S05_Proyecto_alumno.ipynb** desde GitHub → **Open in Colab** → **Copiar en Drive**.

Cargas datos reales de Colpensiones, construyes la base con llaves y restricciones y respondes 8 preguntas con **SQL**; luego integras dos modelos y cargas un mes nuevo para ver todo actualizarse.

Checkpoints: base cargada a las 09:12, ejercicio 7 a las 09:25 y **llegó octubre** (Parte 9) a las 09:38.

Presenta tu hallazgo en 3 minutos

Una cifra del fondo, una cifra real y la consulta que las respondió

1 · La cifra

Un indicador del fondo y uno real del país.

2 · La consulta

El SQL que la respondió, leído en voz alta.

3 · La confianza

Qué regla de calidad o qué alias te preocupa.

4 · La acción

Qué harías en la fuente para que no vuelva a pasar.

Ejemplo: "El 88,9 % de las PQRS se responde a tiempo, bajo la meta del 90 %. *Devolución de aportes* es el peor tema. Lo saqué con un GROUP BY y un HAVING."

Lo que te llevas del programa

Cinco sesiones, un mismo proceso

Explorar

read_csv, info, value_counts: entender el archivo antes de tocarlo.

Limpiar

Texto, fechas, duplicados y merge sobre una copia, nunca sobre el original.

Medir la calidad

Reglas, índice y cuarentena antes de publicar.

Automatizar

Funciones, bucles, bitácora y un reporte que sale solo.

Guardar y mostrar

Modelo formal, SQL, vistas y un tablero para toda la organización.

Siguiente paso

Elige un reporte que repitas cada mes, llévalo a una tabla con llaves y respóndelo con una vista.

"Si lo haces más de dos veces a mano, merece un script; si lo consultan más de dos personas, merece una base."

ean universidad

acreditada
en alta calidad
res. Nº 023654 del Mineducación
10/12/21, vigencia 10/12/27



www.universidadean.edu.co

Universidad Ean: SNIES 2812 Personería Jurídica Res. 2898 del Minjusticia - 16/05/69 - Vigilada Mineducación.

Ean Legacy: Carrera 11 No. 78-47 - Fundadores: Calle 79 # 11 - 45 - Av. Chile: Calle 71 # 9 - 84 Bogotá D.C., Cundinamarca, Colombia, Suramérica.

Línea gratuita nacional: +57-01-8000-931000
Centro de contacto en Bogotá: +(57-601) 593 6161 - +(57-601) 593 6464
E-mail: informacion@universidadean.edu.co

© 2024 Todos los derechos reservados - Universidad Ean